PHARMACY MANAGEMENT SYSTEM

Object-Oriented Programming - Mini Project Report

Submitted in partial fulfillment of the requirements for the
Bachelor of Technology in Computer Science and Engineering

Project Title	Pharmacy Management System
Subject	Object-Oriented Programming (OOP)
Academic Year	2024 - 2025

Team Members

Name	Register Number	Roll No.
Kenneth Martin	240962047	8
Sam Thomas	240962242	18
Mohammed Rayyan Sheik	240962073	12
Vedank Singh	240962043	7

Introduction

The objective of this project is to design and implement a Pharmacy Management System (PMS) using Java and JavaFX to manage medicine inventory, customer records, billing transactions, supplier details, and prescription handling. This system enables pharmacy staff to register and manage medicines in stock, generate bills and invoices for customers, handle prescriptions provided by doctors, and receive automated alerts for low stock or expiring medicines. The system also focuses on ensuring secure data handling through role-based access control and persistent file-based storage using Java Serialization.

Problem Statement

Pharmacy Management System with Inventory Control and Billing Automation

Manual pharmacy operations are heavily dependent on paper-based records, leading to frequent billing errors, poor stock management, and difficulty in tracking medicine availability and expiry. Patients may receive incorrect medicines due to outdated records. There is no automated mechanism to alert staff about low stock or expired medicines. The Pharmacy Management System aims to automate and digitize all core pharmacy operations, improving accuracy, efficiency, and reliability.

Implementation Details

This project was implemented in the Java programming language. The application utilizes object-oriented programming principles to manage pharmacy operations effectively.

The system has the following files, containing the mentioned classes and methods:

- PharmacyApp.java
- LoginScreen.java
- DashboardScreen.java
- Medicine.java
- InventoryManager.java
- BillingService.java
- Customer.java / CustomerService.java
- DataStore.java

Detailed Explanation of Each File

1. PharmacyApp.java

This file serves as the main entry point of the Pharmacy Management System. It extends the JavaFX Application class and overrides the start() method. It initializes all service objects, starts background monitoring threads for expiry and stock alerts, loads sample data on first run, and manages navigation between all screens using Stage.setScene(). This file contains JavaFX codes.

Code for PharmacyApp.java file:

```
package com.pharmacy.ui;
import com.pharmacy.service.*;
import com.pharmacy.model.User;
import javafx.application.Application;
```

```
import javafx.stage.Stage;
import java.io.File;
public class PharmacyApp extends Application {
private UserService userService;
private InventoryManager inventoryManager;
private BillingService billingService;
private ExpiryAlertThread expiryAlertThread;
private Stage primaryStage;
@Override
public void start(Stage primaryStage) {
this.primaryStage = primaryStage;
new File("data/invoices").mkdirs();
userService = new UserService();
inventoryManager = new InventoryManager();
billingService = new BillingService(inventoryManager);
// Thread subclass for expiry alerts
expiryAlertThread = new ExpiryAlertThread(inventoryManager);
expiryAlertThread.setDaemon(true);
expiryAlertThread.start();
// Runnable interface for stock alerts
StockAlertThread stockRunnable = new StockAlertThread(inventoryManager);
Thread stockThread = new Thread(stockRunnable, "StockAlertThread");
stockThread.setDaemon(true);
stockThread.start();
showLoginScreen();
primaryStage.setTitle("PharmaCare - Pharmacy Management System");
primaryStage.show();
}
public void showDashboard(User user) {
DashboardScreen dashboard = new DashboardScreen(this, user);
primaryStage.setScene(dashboard.getScene());
}
public static void main(String[] args) { launch(args); }
}
```

2. LoginScreen.java

This file handles the login interface. The user must enter a valid username and password. The system supports two roles - Admin and Pharmacist. If credentials are incorrect or empty, a red error message is displayed. After successful login, the user is redirected to the Dashboard. The screen uses a GridPane for form layout and PasswordField to mask the password. This file contains JavaFX codes.

Code for LoginScreen.java file:

```
package com.pharmacy.ui;
import com.pharmacy.model.User;
import com.pharmacy.service.UserService;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
```

```
public class LoginScreen {
private Scene scene;
public LoginScreen(PharmacyApp app, UserService userService) {
VBox mainLayout = new VBox(20);
mainLayout.setAlignment(Pos.CENTER);
mainLayout.setStyle("-fx-background-color: #2c3e50;");
Label titleLabel = new Label("PharmaCare");
titleLabel.setFont(Font.font("Arial", FontWeight.BOLD, 36));
titleLabel.setStyle("-fx-text-fill: #ecf0f1;");
GridPane grid = new GridPane(); // GridPane for form layout
grid.setHgap(10); grid.setVgap(15);
grid.setStyle("-fx-background-color: #34495e; -fx-background-radius: 10;");
TextField usernameField = new TextField();
PasswordField passwordField = new PasswordField();
Label messageLabel = new Label();
messageLabel.setStyle("-fx-text-fill: #e74c3c;");
Button loginBtn = new Button("Login");
loginBtn.setOnAction(e -> {
String username = usernameField.getText().trim();
String password = passwordField.getText().trim();
if (username.isEmpty() || password.isEmpty()) {
messageLabel.setText("Username and password cannot be empty.");
return;
}
User user = userService.authenticate(username, password);
if (user != null) app.showDashboard(user);
else messageLabel.setText("Invalid credentials! Please try again.");
});
grid.add(new Label("Username:"), 0, 0); grid.add(usernameField, 1, 0);
grid.add(new Label("Password:"), 0, 1); grid.add(passwordField, 1, 1);
grid.add(loginBtn, 0, 2, 2, 1);
grid.add(messageLabel, 0, 3, 2, 1);
mainLayout.getChildren().addAll(titleLabel, grid);
scene = new Scene(mainLayout, 1100, 700);
}
public Scene getScene() { return scene; }
}
```

3. DashboardScreen.java

This file handles the main dashboard displayed after login. The user can navigate to six modules: Inventory, Billing, Supplier, Customer, Prescription, and Reports. The dashboard shows the logged-in user's name and role in the top bar. A logout button redirects to the login screen. This file contains JavaFX codes.

Code for DashboardScreen.java file:

```
package com.pharmacy.ui;
import com.pharmacy.model.User;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
```

```
public class DashboardScreen {
private Scene scene;
public DashboardScreen(PharmacyApp app, User currentUser) {
BorderPane root = new BorderPane();
HBox topBar = new HBox(20);
topBar.setStyle("-fx-background-color: #2c3e50;");
Label title = new Label("PharmaCare Dashboard");
title.setFont(Font.font("Arial", FontWeight.BOLD, 22));
title.setStyle("-fx-text-fill: white;");
Region spacer = new Region();
HBox.setHgrow(spacer, Priority.ALWAYS);
Button logoutBtn = new Button("Logout");
logoutBtn.setOnAction(e -> app.showLoginScreen());
topBar.getChildren().addAll(title, spacer, logoutBtn);
GridPane moduleGrid = new GridPane(); // GridPane for module layout
moduleGrid.setAlignment(Pos.CENTER);
moduleGrid.setHgap(25); moduleGrid.setVgap(25);
Button inventoryBtn = createModuleButton("Inventory Management", "#3498db");
Button billingBtn = createModuleButton("Sales and Billing", "#27ae60");
Button supplierBtn = createModuleButton("Supplier Management", "#e67e22");
Button customerBtn = createModuleButton("Customer Records", "#9b59b6");
Button presBtn = createModuleButton("Prescription Handling","#1abc9c");
Button reportsBtn = createModuleButton("Reports and Alerts", "#e74c3c");
inventoryBtn.setOnAction(e -> app.showInventoryScreen());
billingBtn.setOnAction(e -> app.showBillingScreen());
moduleGrid.add(inventoryBtn,0,0); moduleGrid.add(billingBtn,1,0);
moduleGrid.add(supplierBtn,2,0); moduleGrid.add(customerBtn,0,1);
moduleGrid.add(presBtn,1,1); moduleGrid.add(reportsBtn,2,1);
root.setTop(topBar); root.setCenter(moduleGrid);
scene = new Scene(root, 1100, 700);
}
private Button createModuleButton(String title, String color) {
Button btn = new Button(title);
btn.setPrefSize(230, 130);
btn.setStyle("-fx-background-color:" + color + ";-fx-text-fill:white;" +
"-fx-font-size:15;-fx-background-radius:10;");
return btn;
}
public Scene getScene() { return scene; }
}
```

4. Medicine.java

This file handles medicine information. It implements Serializable for data persistence and Comparable for sorting by name. It demonstrates Encapsulation through private fields, Wrapper classes (Integer, Double) with autoboxing/unboxing, and constructor overloading.

Code for Medicine.java file:

```
package com.pharmacy.model;
import java.io.Serializable;
public class Medicine implements Serializable, Comparable<Medicine> {
private static final long serialVersionUID = 1L;
private String medicineId, name, manufacturer, batchNumber, expiryDate, supplierId;
private MedicineCategory category; // Enum
```

```
private Integer quantity; // Wrapper class (autoboxing)
private Double price; // Wrapper class (autoboxing)
// Parameterized constructor
public Medicine(String medicineId, String name, String manufacturer,
MedicineCategory category, String batchNumber,
String expiryDate, int quantity, double price, String supplierId) {
this.medicineId = medicineId; this.name = name;
this.manufacturer = manufacturer; this.category = category;
this.batchNumber = batchNumber; this.expiryDate = expiryDate;
this.quantity = quantity; // Autoboxing: int -> Integer
this.price = price; // Autoboxing: double -> Double
this.supplierId = supplierId;
}
// Constructor overloading (without supplierId)
public Medicine(String medicineId, String name, String manufacturer,
MedicineCategory category, String batchNumber,
String expiryDate, int quantity, double price) {
this(medicineId, name, manufacturer, category, batchNumber, expiryDate, quantity, price,
"");
}
@Override
public int compareTo(Medicine other) {
return this.name.compareToIgnoreCase(other.name); // Comparable for Collections.sort()
}
// Encapsulation - Getters and Setters
public String getMedicineId() { return medicineId; }
public String getName() { return name; }
public Integer getQuantity() { return quantity; }
public Double getPrice() { return price; }
public int getQuantityPrimitive() { return quantity; } // Unboxing
public void setQuantity(Integer qty) { this.quantity = qty; }
public void setPrice(Double p) { this.price = p; }
@Override
public String toString() {
return medicineId + " | " + name + " | Qty:" + quantity
+ " | Rs." + String.format("%.2f", price) + " | Exp:" + expiryDate;
}
}
```

5. InventoryManager.java

This file manages the medicine inventory. It uses an ArrayList for sequential storage and a HashMap for O(1) ID-based lookup. The Iterator is used during deletion to avoid ConcurrentModificationException. Collections.sort() sorts medicines by name, price, or expiry date. All critical methods are synchronized for thread safety.

Code for InventoryManager.java file:

```
package com.pharmacy.service;
import com.pharmacy.model.Medicine;
import com.pharmacy.util.DataStore;
import java.util.*;
public class InventoryManager {
private ArrayList<Medicine> medicineList;
private HashMap<String, Medicine> medicineMap;
private DataStore<Medicine> dataStore;
```

```
private static final int LOW_STOCK_THRESHOLD = 10;
public InventoryManager() {
dataStore = new DataStore<>("data/medicines.dat");
medicineList = dataStore.loadAll();
medicineMap = new HashMap<>();
for (Medicine med : medicineList)
medicineMap.put(med.getMedicineId(), med);
}
public synchronized void addMedicine(Medicine medicine) {
medicineList.add(medicine);
medicineMap.put(medicine.getMedicineId(), medicine);
dataStore.saveAll(medicineList);
}
// Delete using Iterator (safe removal)
public synchronized void deleteMedicine(String id) {
Iterator<Medicine> it = medicineList.iterator();
while (it.hasNext()) {
if (it.next().getMedicineId().equals(id)) { it.remove(); break; }
}
medicineMap.remove(id);
dataStore.saveAll(medicineList);
}
public Medicine getMedicineById(String id) { return medicineMap.get(id); }
public ArrayList<Medicine> getSortedByName() {
ArrayList<Medicine> sorted = new ArrayList<>(medicineList);
Collections.sort(sorted); // Uses Comparable
return sorted;
}
public ArrayList<Medicine> getSortedByPrice() {
ArrayList<Medicine> sorted = new ArrayList<>(medicineList);
Collections.sort(sorted, (m1, m2) -> Double.compare(m1.getPrice(), m2.getPrice()));
return sorted;
}
public ArrayList<Medicine> getLowStockMedicines() {
ArrayList<Medicine> low = new ArrayList<>();
for (Medicine m : medicineList)
if (m.getQuantity() < LOW_STOCK_THRESHOLD) low.add(m);
return low;
}
public synchronized boolean reduceStock(String id, int qty) {
Medicine med = medicineMap.get(id);
if (med != null && med.getQuantity() >= qty) {
med.setQuantity(med.getQuantity() - qty);
dataStore.saveAll(medicineList);
return true;
}
return false;
}
public ArrayList<Medicine> getAllMedicines() { return medicineList; }
public int getMedicineCount() { return medicineList.size(); }
}
```

6. BillingService.java

This file manages all billing operations. When a bill is created, stock is reduced for each medicine and the bill is saved. A formatted invoice text file is generated using FileWriter. Billing is processed in a separate BillingThread to keep the UI responsive. The Bill class demonstrates method overloading with two versions of applyDiscount().

Code for BillingService.java file:

```
package com.pharmacy.service;
import com.pharmacy.model.Bill;
import com.pharmacy.model.BillItem;
import com.pharmacy.util.DataStore;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
public class BillingService {
private ArrayList<Bill> billList;
private DataStore<Bill> dataStore;
private InventoryManager inventoryManager;
public BillingService(InventoryManager inventoryManager) {
this.inventoryManager = inventoryManager;
dataStore = new DataStore<>("data/bills.dat");
billList = dataStore.loadAll();
}
public synchronized boolean createBill(Bill bill) {
for (BillItem item : bill.getItems()) {
boolean ok = inventoryManager.reduceStock(item.getMedicineId(), item.getQuantity());
if (!ok) return false;
}
billList.add(bill);
dataStore.saveAll(billList);
generateInvoice(bill);
return true;
}
public void generateInvoice(Bill bill) { // FileWriter for invoice
String path = "data/invoices/" + bill.getBillId() + "_invoice.txt";
try (FileWriter writer = new FileWriter(path)) {
writer.write("===== PHARMACARE INVOICE =====
");
writer.write("Invoice No : " + bill.getBillId() + "
");
writer.write("Customer : " + bill.getCustomerName() + "
");
writer.write("Date : " + bill.getDate() + "
");
writer.write("-----------------------------
");
for (BillItem item : bill.getItems())
writer.write(String.format("%-18s x%d = Rs.%.2f
",
item.getMedicineName(), item.getQuantity(), item.getSubtotal()));
writer.write("-----------------------------
");
writer.write(String.format("Net Amount : Rs.%.2f
", bill.getNetAmount()));
} catch (IOException e) { System.err.println("Invoice error: " + e.getMessage()); }
```

```
}
// Method overloading
public void applyDiscount(Bill bill, double percentage) { bill.applyDiscount(percentage);
}
public void applyDiscount(Bill bill, double amount, boolean isFixed) {
bill.applyDiscount(amount, isFixed); }
public double getTotalSales() {
double total = 0;
for (Bill b : billList) total += b.getNetAmount();
return total;
}
public ArrayList<Bill> getAllBills() { return billList; }
}
```

7. Customer.java / CustomerService.java

These files handle customer information. Customer.java stores customer ID, name, phone, email, and address. CustomerService.java manages all CRUD operations using ArrayList for storage and HashMap for fast ID-based lookups. Iterator is used for safe deletion. Data is persisted via the generic DataStore class.

Code for Customer.java file:

```
package com.pharmacy.model;
import java.io.Serializable;
public class Customer implements Serializable {
private static final long serialVersionUID = 1L;
private String customerId, name, phone, email, address;
public Customer(String customerId, String name, String phone,
String email, String address) {
this.customerId = customerId; this.name = name;
this.phone = phone; this.email = email; this.address = address;
}
public String getCustomerId() { return customerId; }
public String getName() { return name; }
public String getPhone() { return phone; }
public void setName(String n) { this.name = n; }
@Override
public String toString() { return customerId + " | " + name + " | " + phone; }
}
```

Code for CustomerService.java file:

```
package com.pharmacy.service;
import com.pharmacy.model.Customer;
import com.pharmacy.util.DataStore;
import java.util.*;
public class CustomerService {
private ArrayList<Customer> customerList;
private HashMap<String, Customer> customerMap;
private DataStore<Customer> dataStore;
public CustomerService() {
dataStore = new DataStore<>("data/customers.dat");
customerList = dataStore.loadAll();
customerMap = new HashMap<>();
for (Customer c : customerList) customerMap.put(c.getCustomerId(), c);
}
```

```
public synchronized void addCustomer(Customer customer) {
customerList.add(customer);
customerMap.put(customer.getCustomerId(), customer);
dataStore.saveAll(customerList);
}
public synchronized void deleteCustomer(String id) { // Iterator for safe delete
Iterator<Customer> it = customerList.iterator();
while (it.hasNext()) {
if (it.next().getCustomerId().equals(id)) { it.remove(); break; }
}
customerMap.remove(id);
dataStore.saveAll(customerList);
}
public Customer getCustomerById(String id) { return customerMap.get(id); } // O(1) lookup
public ArrayList<Customer> searchByName(String name) {
ArrayList<Customer> results = new ArrayList<>();
for (Customer c : customerList)
if (c.getName().toLowerCase().contains(name.toLowerCase())) results.add(c);
return results;
}
public ArrayList<Customer> getAllCustomers() { return customerList; }
}
```

8. DataStore.java

This file handles all data persistence. It is a generic class (DataStore<T extends Serializable>) reusable across all service classes. It uses ObjectOutputStream and FileOutputStream to serialize (save) objects, and ObjectInputStream and FileInputStream to deserialize (load) them. FileWriter creates text backups and FileReader reads them. All methods are synchronized for thread safety.

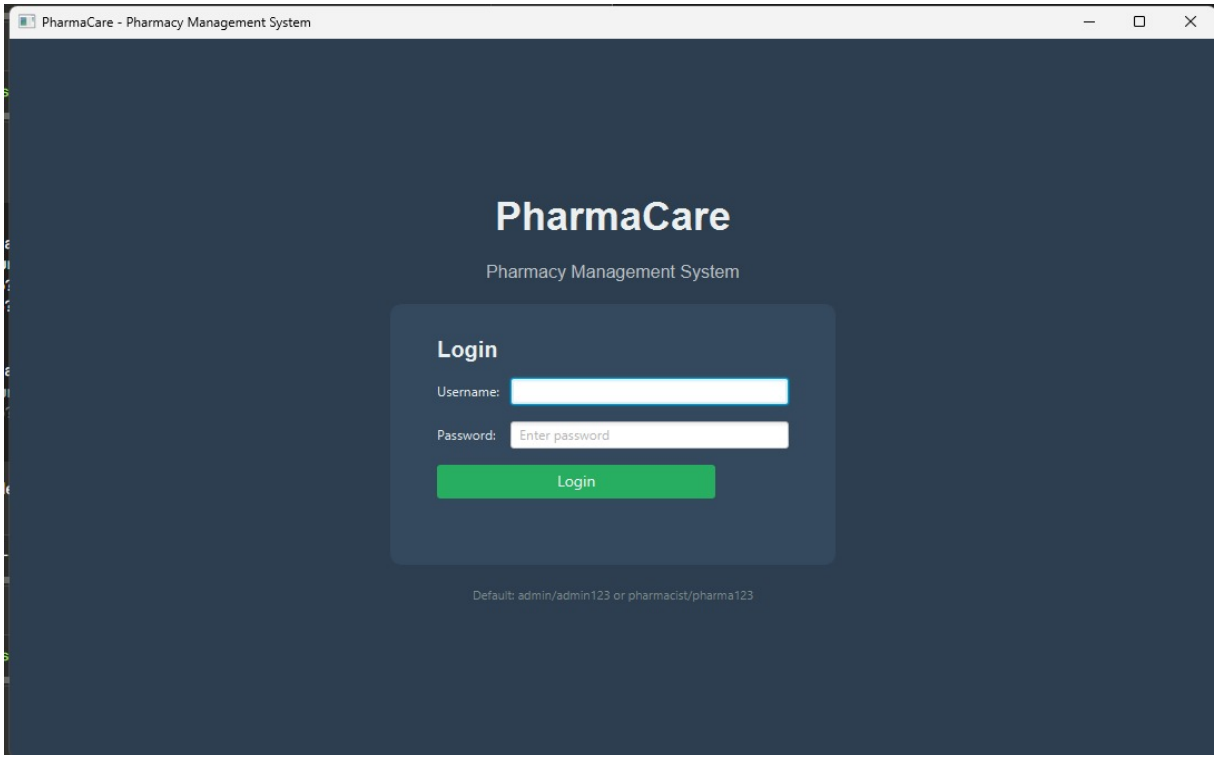
Code for DataStore.java file:

```
package com.pharmacy.util;
import java.io.*;
import java.util.ArrayList;
public class DataStore<T extends Serializable> { // Generic class
private String filePath;
public DataStore(String filePath) { this.filePath = filePath; }
// Serialization using ObjectOutputStream + FileOutputStream
public synchronized void saveAll(ArrayList<T> items) {
try (FileOutputStream fos = new FileOutputStream(filePath);
ObjectOutputStream oos = new ObjectOutputStream(fos)) {
oos.writeObject(items);
} catch (IOException e) { System.err.println("Save error: " + e.getMessage()); }
}
// Deserialization using ObjectInputStream + FileInputStream
@SuppressWarnings("unchecked")
public synchronized ArrayList<T> loadAll() {
File file = new File(filePath);
if (!file.exists()) return new ArrayList<>();
try (FileInputStream fis = new FileInputStream(filePath);
ObjectInputStream ois = new ObjectInputStream(fis)) {
return (ArrayList<T>) ois.readObject();
} catch (IOException | ClassNotFoundException e) { return new ArrayList<>(); }
}
```

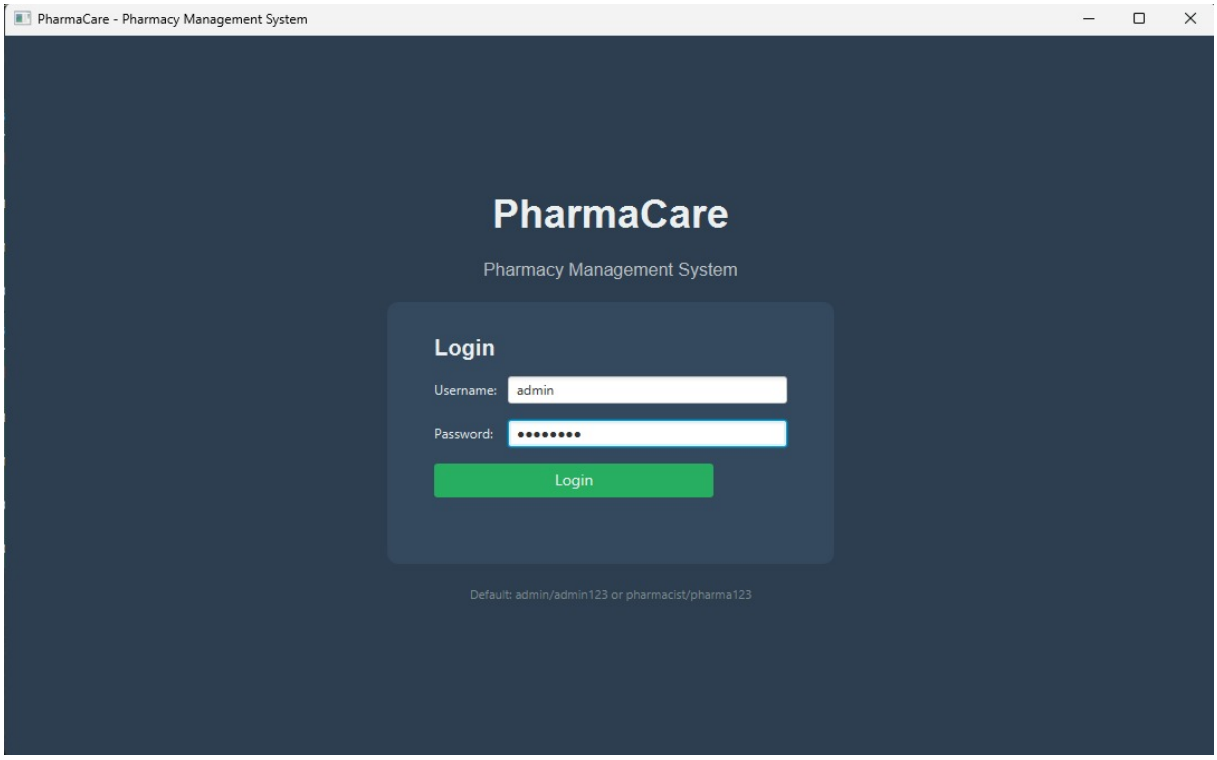
```
// Text backup using FileWriter
public void backupToText(ArrayList<T> items, String backupPath) {
try (FileWriter writer = new FileWriter(backupPath)) {
writer.write("=== BACKUP: " + filePath + " ===
");
for (T item : items) writer.write(item.toString() + "
");
} catch (IOException e) { System.err.println("Backup error: " + e.getMessage()); }
}
// Read backup using FileReader
public String readBackup(String backupPath) {
StringBuilder sb = new StringBuilder();
try (FileReader reader = new FileReader(backupPath)) {
int ch;
while ((ch = reader.read()) != -1) sb.append((char) ch);
} catch (IOException e) { return "No backup found."; }
return sb.toString();
}
}
```

Results

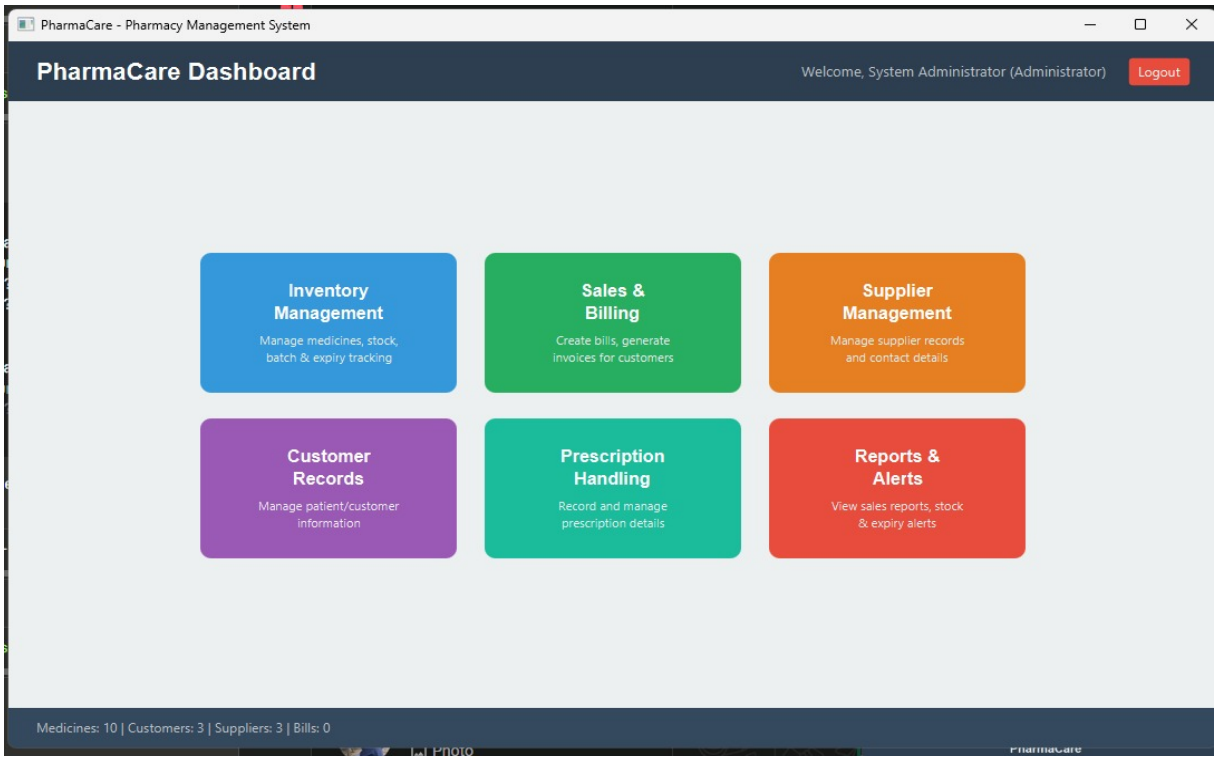
Login Screen



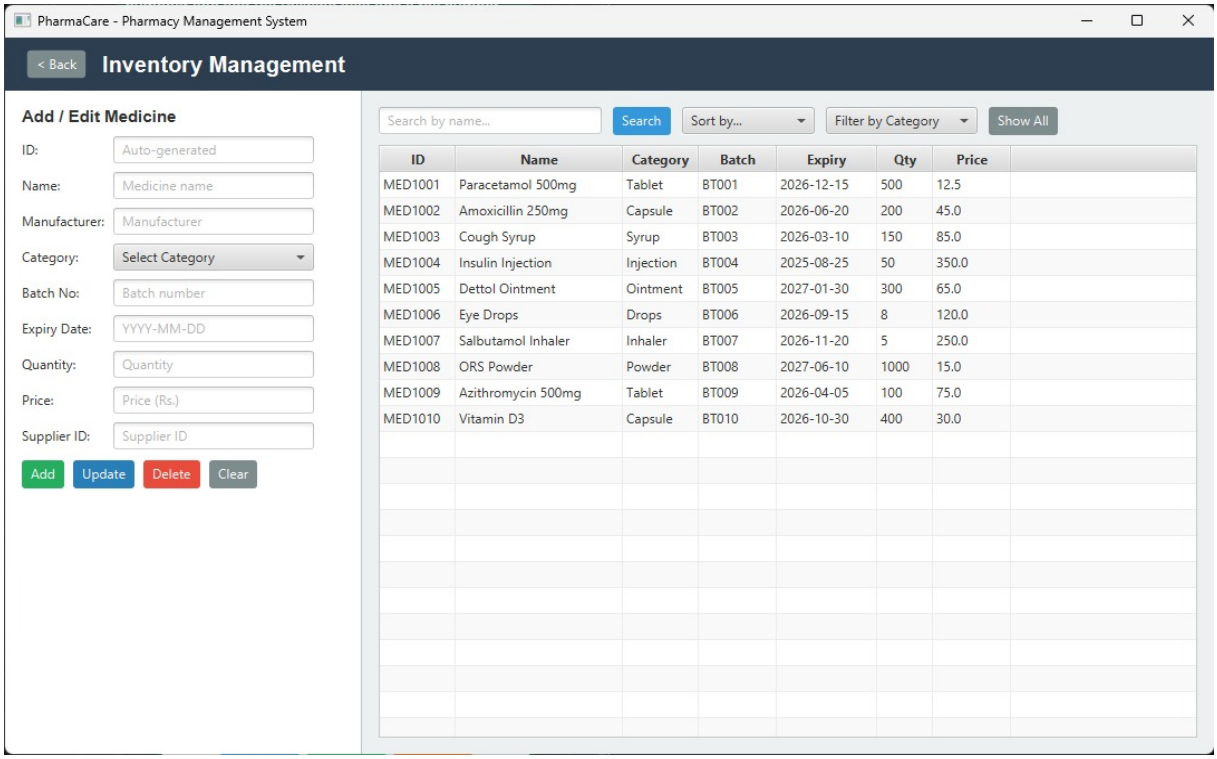
Login Screen - with Credentials



Dashboard Screen

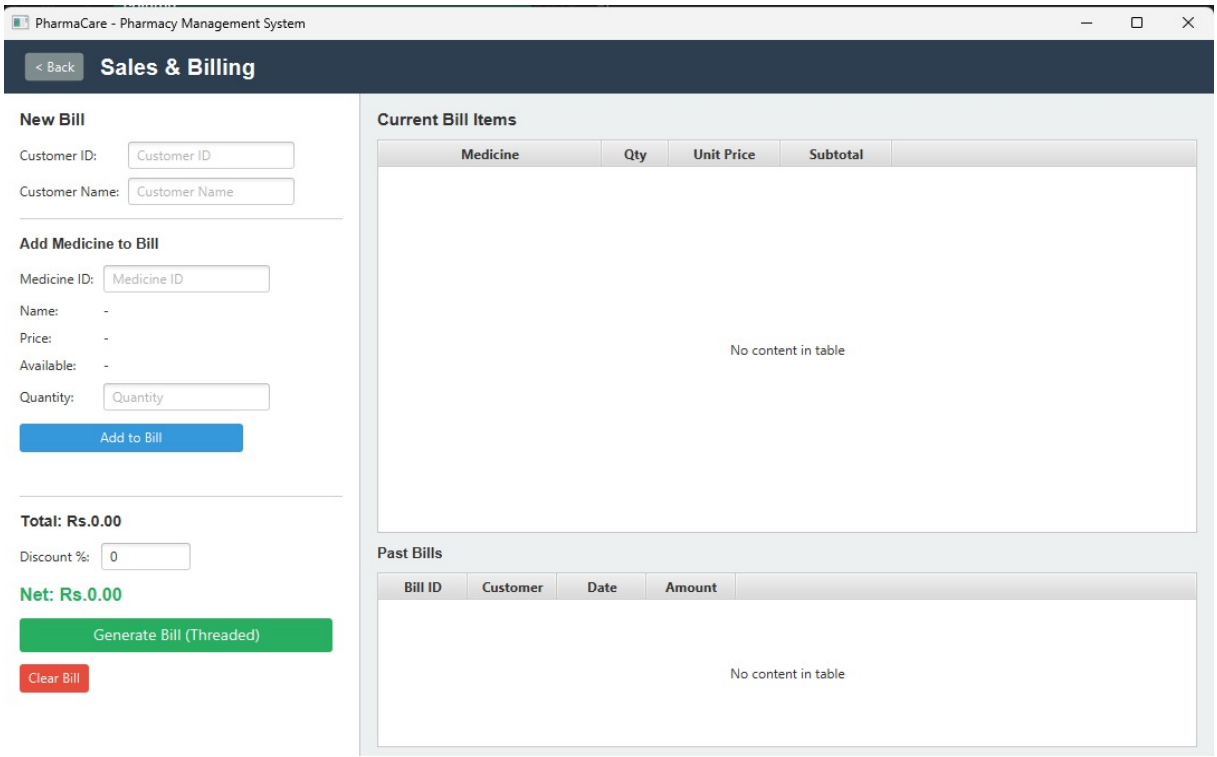


Inventory Management Screen

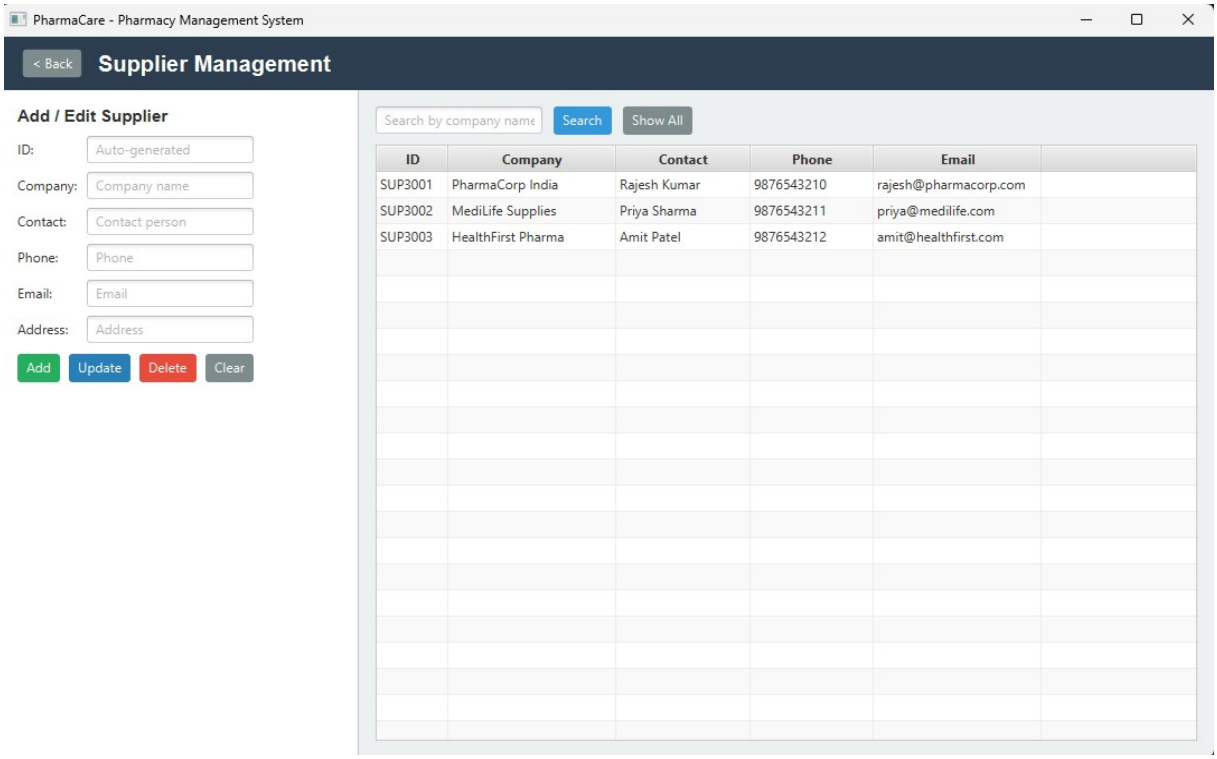


ID	Name	Category	Batch	Expiry	Qty	Price
MED1001	Paracetamol 500mg	Tablet	BT001	2026-12-15	500	12.5
MED1002	Amoxicillin 250mg	Capsule	BT002	2026-06-20	200	45.0
MED1003	Cough Syrup	Syrup	BT003	2026-03-10	150	85.0
MED1004	Insulin Injection	Injection	BT004	2025-08-25	50	350.0
MED1005	Dettol Ointment	Ointment	BT005	2027-01-30	300	65.0
MED1006	Eye Drops	Drops	BT006	2026-09-15	8	120.0
MED1007	Salbutamol Inhaler	Inhaler	BT007	2026-11-20	5	250.0
MED1008	ORS Powder	Powder	BT008	2027-06-10	1000	15.0
MED1009	Azithromycin 500mg	Tablet	BT009	2026-04-05	100	75.0
MED1010	Vitamin D3	Capsule	BT010	2026-10-30	400	30.0

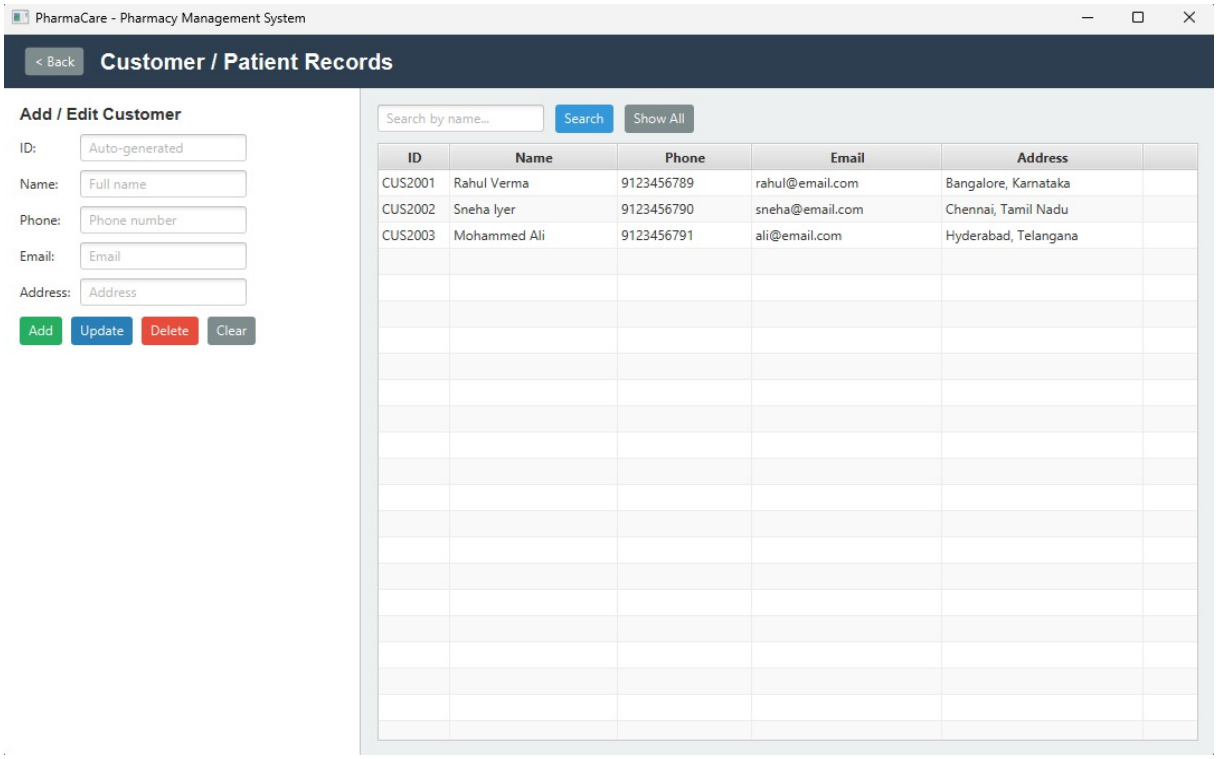
Sales and Billing Screen



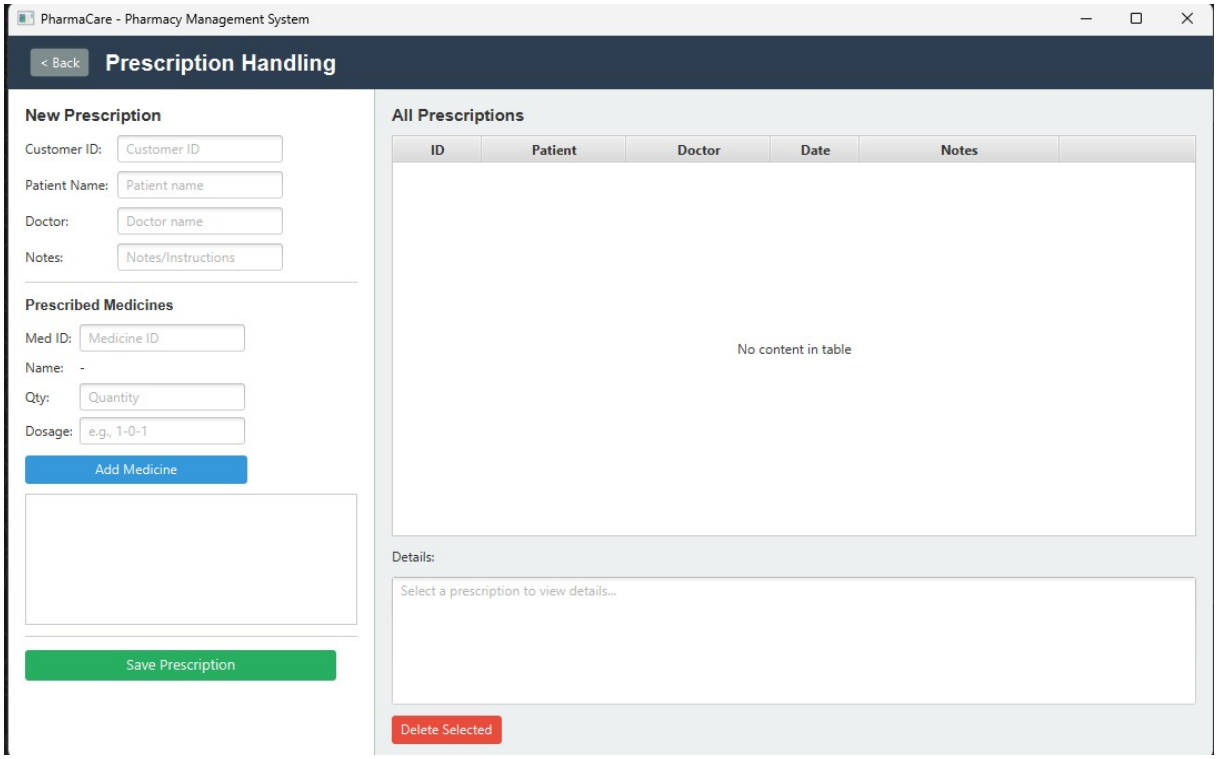
Supplier Management Screen



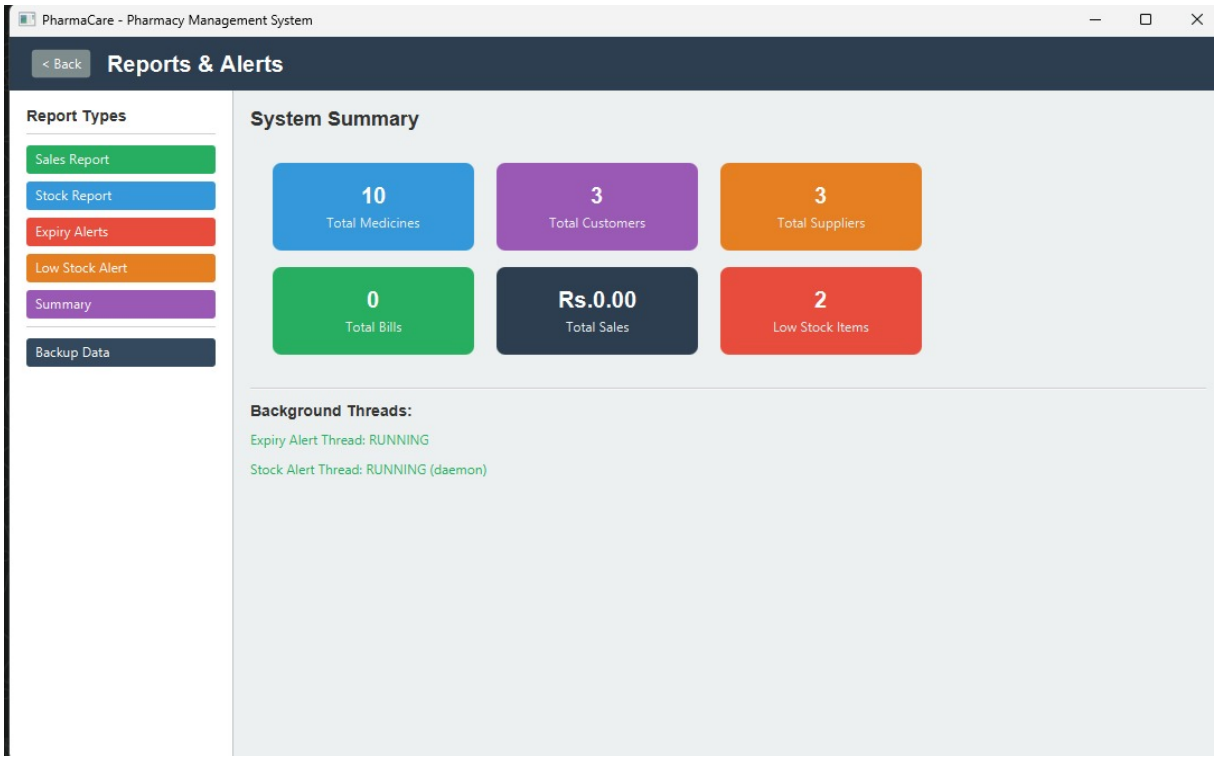
Customer Records Screen



Prescription Handling Screen



Reports and Alerts Screen


PharmaCare - Pharmacy Management System
< Back
Reports & Alerts
Report Types
Sales Report
Stock Report
Expiry Alerts
Low Stock Alert
Summary
Backup Data
System Summary
10
Total Medicines
3
Total Customers
3
Total Suppliers
0
Total Bills
Rs.0.00
Total Sales
2
Low Stock Items
Background Threads:
Expiry Alert Thread: RUNNING
Stock Alert Thread: RUNNING (daemon)

Conclusion

The Pharmacy Management System (PMS) provides an efficient and reliable platform for pharmacy staff to manage medicine inventory, process billing transactions, maintain customer and supplier records, handle prescriptions, and generate automated alerts for expiring or low-stock medicines. Using Java and JavaFX, the system delivers a clean, intuitive graphical interface that makes it easy for both administrators and pharmacists to interact with their data securely and accurately.

By integrating role-based access control, unique auto-generated IDs for each record, and synchronized thread-safe service methods, the PMS ensures that data integrity is preserved even under concurrent access conditions. The use of background threads for expiry and stock monitoring, combined with automatic invoice generation using FileWriter, makes the system both proactive and practical for day-to-day pharmacy operations.

Overall, the PMS successfully demonstrates how object-oriented programming principles - Encapsulation, Inheritance, Polymorphism, and Abstraction - along with Java Collections, Generics, Multithreading, File Handling, and Serialization can be applied to solve real-world problems in healthcare management, resulting in a secure, modular, and scalable solution.

Individual Contributions

Kenneth Martin, 240962047, Roll No. 8

- PharmacyApp.java
- LoginScreen.java
- DashboardScreen.java
- UserService.java
- User.java / UserRole.java

Sam Thomas, 240962242, Roll No. 18

- Medicine.java / MedicineCategory.java
- InventoryManager.java
- InventoryScreen.java
- SampleDataLoader.java

Mohammed Rayyan Sheik, 240962073, Roll No. 12

- BillingService.java
- BillingThread.java
- BillingScreen.java
- Bill.java / BillItem.java

Vedank Singh, 240962043, Roll No. 7

- DataStore.java
- IDGenerator.java
- CustomerService.java / SupplierService.java
- PrescriptionService.java
- ExpiryAlertThread.java / StockAlertThread.java
- BackupService.java
- ReportsScreen.java

While project tasks were divided among the team members, we collaborated closely at every stage, discussing and refining our design decisions together. Since this system strictly follows Object-Oriented Programming principles, each file is interdependent, requiring continuous communication and coordination to ensure seamless integration across all classes and modules. By regularly reviewing each other's work and resolving challenges collaboratively, we maintained consistency in design and functionality, resulting in a unified and robust Pharmacy Management System.

References

- 1. JavaFX Official Documentation: https://openjfx.io/
- 2. JavaFX Tutorials and Codes: https://www.geeksforgeeks.org/tag/javafx/
- 3. Java Serialization Guide: https://www.baeldung.com/java-serialization
- 4. Java Collections Framework: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html
- 5. Java Multithreading: https://www.geeksforgeeks.org/multithreading-in-java/